JavaScript

Presented By: Taruna Sharma

Topics to be covered

1. Introduction to Javascript
2. Scope
3. Closure
4. IIFE
5. This keyword
6. Call ,Bind, Apply

Introduction to Javascript

- **JavaScript** is a cross-platform, scripting language used to make webpages interactive.
- ECMASCRIPT is the official name of the language.
- ECMASCRIPT versions have been abbreviated to ES1, ES2, ES3, ES5 and ES6.
- ES6 is backward compatible that means, developer can start with older version syntax and eventually using ES6 in between, will not break the code.
- Javascript is a synchronous single-threaded language.

Scope

In JavaScript, there are 3 ways to declare a variable: `let`, `const` (the modern ones), and `var` (the remnant of the past).

- Variables declared with let and const behave the same.
- The old `var` has some notable differences.

Before, ES6, Javascript had only two types of scope.

1.**GLobal Scope**: Variables declared globally(outside any function) can be accessed from anywhere in a Javascript program.

2.**Function Scope**: Variables declared Locally(inside any function) can be accessed only inside the function where they are declared.

LET Keyword

Block Scope: If a variable is declared inside a code block than it's visible only inside that block.

They cannot be accessed outside the block.

We can use the block scope, to isolate piece of code that does its own task.

```
/* this piece of code isolaltely does
its own task, with variables that only belong to it:*/
{
let myName = "Taruna";
console.log(myName);
}
{
let myName = "hello";
console.log(myName);
}
console.log(myName);  //error variable isn't defined
```

What if we declare the same global variable inside code block..

Will the change in code block value effects the global value ?

```
/*2.let Variable can be declared outside as well.
myName value remains undefined even if the value
has been assigned inside the block.*/
let myName;
{
let myName = "Taruna";
console.log(myName);
}


console.log(myName);
```

:

Output: Taruna

: Undefined

But If we do not declare the variable inside and only change the value there than what?

```
let myName;
{
        myName = "Taruna";
        console.log(myName);
}


console.log(myName);
```

Output: Taruna

: Taruna

The change in value reflects in the global variable also.This happened because of the LEXICAL SCOPE. Lets understand what is lexical scope now.

LEXICAL ENVIRONMENT

In JavaScript, every running function, code block `{...}`, and the script as a whole have an internal (hidden) associated object known as the *Lexical Environment*.

The Lexical Environment object consists of two parts:

1. *Environment Record* – an object that stores all local variables as its properties (and some other information like the value of `this`).
2. A reference to the *outer lexical environment*, the one associated with the outer code.

When a function runs, at the beginning of the call, a new Lexical Environment is created automatically to store local variables and parameters of the call.

When the code wants to access a variable – the inner Lexical Environment is searched first, then the outer one, then the more outer one and so on until the global one.

The changes are made in the respective lexical environment only.

On page 6 Code the myName variable is declared inside the block hence the changes are made there itself whereas in the other code the changes are made globally since the variable is not declared inside the block.

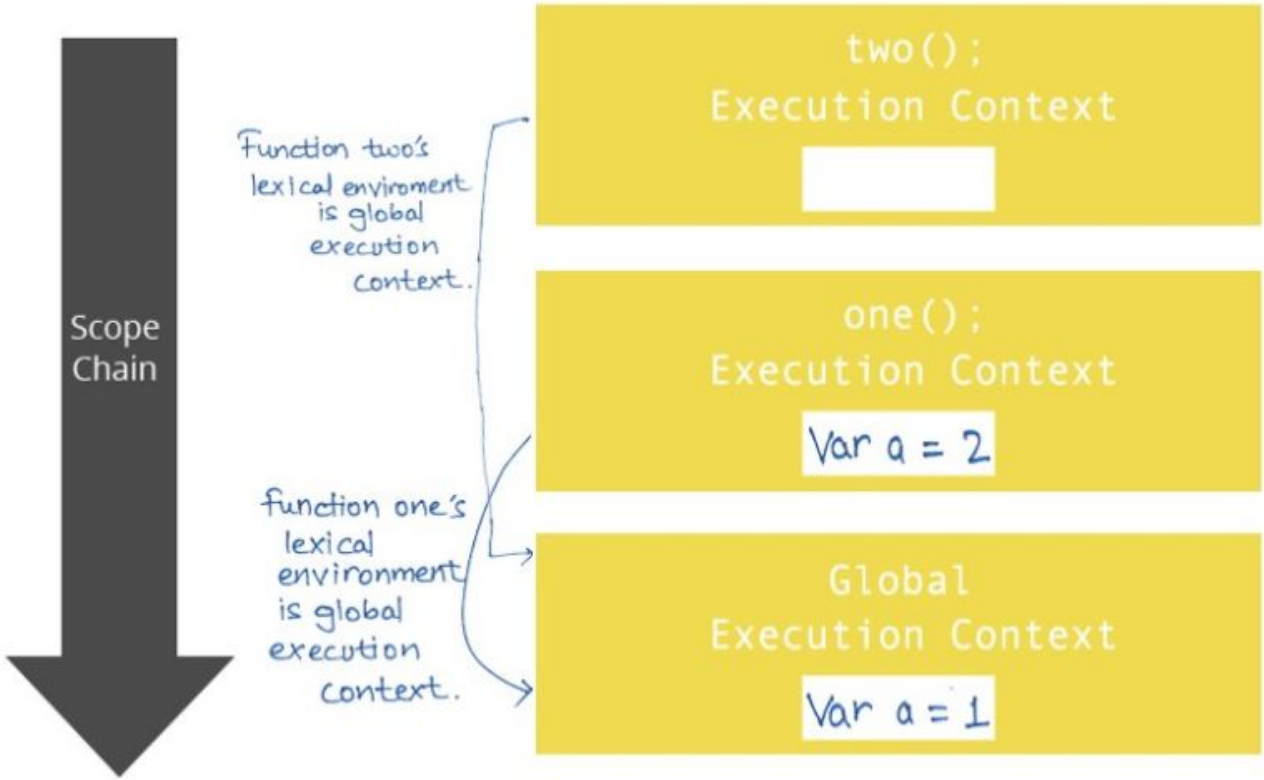
Predict the output

```
function two(){
console.log(a);
}

function one(){
let a= 2;
console.log(a);
two();
}

let a=1;
console.log(a);
one();
```

Output: 1

2

1



Execution stack showing the lexical environment for function one and function two.

The old “var”

Points to be noted:

1. **Var has no block level scope.Variables, declared with `var`, are either function-scoped or global-scoped.**

.

```
//var has no block level scope
function sayHi() {
  if (true) {
    var phrase = "Hello";
  }

  alert(phrase); // works outside if block
}

sayHi();
alert(phrase);  //error: function level scope only
```

If a code block is inside a function, then `var` becomes a function-level variable.

2. **Unlike let, ‘Var’ tolerates re-decleration.**

```
//var tolerates redecleration

var user = "Pete";

var user = "John"; // it doesn't trigger an error


console.log(user); // John
```

3. **‘Var ’ variables can be declared below their use.This concept is called hoisting.**

HOISTING :

`var` declarations are processed when the function starts (or script starts for globals).

```
//var can be declared below thier use.
//hoisting--
function sayHi() {
  phrase = "Hello";

  console.log(phrase);

  var phrase;
}
sayHi();

//all var are raised  to the top of the function.
```

Output: “hello”

So in the example , the var declaration inside the function is processed in the beginning of the function, so at the moment of logging the variable exists.

No ‘hoisting’ in case of let variables.

In case of ‘let’ variable a we can’t assign a value to a variable before it’s declaration.

```
function sayHi() {
        phrase = "Hello";
  console.log(phrase);
  let phrase;
}

sayHi();      //ReferenceError
```

‘Temporal dead zone’ : The `let` and `const` variables exist in the TDZ from the start of their enclosing scope until they are declared.

Point to be noted in Hoisting.

Declarations are hoisted, but assignments are not.

```
function sayHi() {
  alert(phrase);
        var phrase = "Hello";
}

sayHi();      //undefined
```

The line var phrase = “hello” has 2 parts:

Variable declaration: var phrase; and variable assignment: phrase= “hello”;

The declaration is processed at the start of function execution (“hoisted”), but the assignment always works at the place where it appears.

4. **At the top level, let, unlike var, does not create a property on the global object:**

```
var foo = "Foo";
let bar = "Bar";

console.log(window.foo); // Foo
console.log(window.bar); // undefined
```

GLOBAL OBJECT

The global object holds variables that should be available everywhere.

In-browser, unless we're using modules, global functions and variables declared with `var` become a property of the global object.

To summarize,

Variables declared by `var` keyword are scoped to the immediate function body (hence the function scope) while `let` variables are scoped to the immediate *enclosing* block denoted by { } (hence the block scope).

These are the main differences of `var` compared to `let/const`:

1. `var` variables have no block scope, their visibility is scoped to current function, or global, if declared outside function.
2. `var` tolerates re-decleration.
3. `var` declarations are processed at function start (script start for globals). I.e. Hoisting.
4. `var` if declared globally becomes the property of global object(window) whereas it does not happen in case of let variables.

.

Closures:

The inner function can access the variable of the enclosing function due to closures in javascript.

In other words, an inner function preserves(remembers) the scope chain of enclosing function at the time the enclosing function was executed, and thus can access the enclosing function's variables.

```
function makeCounter() {
  let count = 0;

  let a = function() {
    return count++;
  };
   return a;
}

let counter = makeCounter();
console.log( counter() ); // 0
console.log( counter() ); // 1
console.log( counter() ); // 2
```

What happens if we create multiple counters??

```
function makeCounter() {
  let count = 0;
    return function() {
    return count++;
  };
}
let counter = makeCounter();
let counter2 = makeCounter();
console.log( counter() ); // 0
console.log( counter() ); //1
console.log( counter2() );
console.log( counter2() );   //  ??  0,1 or 1,2
```

Output: 0,1

Functions `counter` and `counter2` are created by different invocations of `makeCounter`.

So they have independent outer Lexical Environments, each one has its own `count`.

IIFE(Immediately Invoked Function Expressions)

IIFE is a function expression that automatically invokes after completion of the definition.

In the past, as there was only var, and it has no block-level visibility, programmers invented a way to emulate it.

1. IIFE does not have a function name.
2. IIFE are called as soon as they are created.
3. IIFE provides a block level scope.

```
//immediately invoked function expression-
var foo = "foo";

(function(){
        var foo = "foo2";//giving foo a blockscope
  console.log(foo);
})();

console.log(foo);
```

Output: foo2

foo

Ways to create IIFE :-

There are basically 4 ways to create IIFE.

```
1.    (function() {
        alert("Parentheses around the function");
      })();

2.    (function() {
        alert("Parentheses around the whole thing");
      }());

3.    !function() {
        alert("Bitwise NOT operator starts the expression");
      }();

4.    +function() {
        alert("Unary plus starts the expression");
      }();
```

This keyword

Every function, while executing, has a reference to its current execution context(where and how the function is called), called “this”. It does not matter where function is declared or defined.

Default Binding

1. Alone this refers to the global object.

```
console.log(this);  //refers to window

var b = {
  name: "penny",
        context: this
}


console.log(b.context===window); //true
```

2. In strict mode, this points to undefined since global object is not eligible for default binding.

```
'use strict'
function  foo(){
        console.log(this.bar);
}

var bar = "bar1";
foo();      //error: this is undefined
```

Implicit Binding

The implicit binding rule states that , when there is a function object for context, than its that object which should be used for function binding.

```
function  foo(){
        console.log(this.bar);
}

var obj = { bar : "heya" , foo:foo };
var obj2 = { bar : "hello" , foo:foo };
var bar = "bar1";

foo();       //bar1
obj.foo();   //heya
obj2.foo(); //hello
```

Output: bar1 heya hello

Explicit binding

Call(), Apply(), bind() methods

1. bind():

The **bind()** method creates a new function that, when called, has its this keyword set to the provided value.

This is extremely powerful. It let's us explicitly define the value of this when calling a function.

2. call(), apply()

The **call(), apply()** method calls a function with a given this value and arguments provided individually.

What that means, is that we can call any function, and *explicitly specify what this should reference* within the calling function. Really similar to the bind() method!

We will see the differences later.

bind() Example

JavaScript + No-Library (pure JS) ▼

```
var pokemon = {
    firstname: 'Pika',
    lastname: 'Chu ',
    getPokeName: function() {
        var fullname = this.firstname + ' ' + this.lastname;
        return fullname;
    }
};

var pokemonName = function() {
    console.log(this.getPokeName() + 'I choose you!');
};

var logPokemon = pokemonName.bind(pokemon); // creates new object and binds pokemon. 'this'
of pokemon === pokemon now

logPokemon(); // 'Pika Chu I choose you!'

```

The main differences between `bind()` and `call()` is that the `call()` method:

1. Accepts additional parameters as well
2. Executes the function it was called upon right away.
3. The `call()` method does not make a copy of the function it is being called on.

`call()` and `apply()` serve the **exact same purpose.** The ***only difference between how they work is that*** call() expects all parameters to be passed in individually, whereas apply() expects an array of all of our parameters.

JavaScript + No-Library (pure JS) ▼

```javascript
var pokemon = {
    firstname: 'Pika',
    lastname: 'Chu ',
    getPokeName: function() {
        var fullname = this.firstname + ' ' + this.lastname;
        return fullname;
    }
};

var pokemonName = function(snack, hobby) {
    console.log(this.getPokeName() + ' loves ' + snack + ' and ' + hobby);
};

pokemonName.call(pokemon,'sushi', 'algorithms'); // Pika Chu  loves sushi and algorithms
pokemonName.apply(pokemon,['sushi', 'algorithms']); // Pika Chu  loves sushi and algorithms
```

NEW BINDING

```
// New binding
function foo(a) {
 this.a = a;
}
var bar = new foo(2);
console.log(bar.a);         // 2
```

By calling foo(), with new in front of it, we have constructed a new object and set that new object as this for the call of foo(..).

This is called the new binding.

EXCEPTION...

Normal functions abide by the four rules we just covered.But ES6 introduced a special kind of functions that does not abide by these rules: Arrow Functions.

Arrow functions are not signified by the function keyword but by the => “fat-arrow” operator.

Instead of using the four standard rules, arrow function adopt the this binding from the enclosing (function or global) scope.

```
// Arrow function and lexical this

function foo() {

 // return an arrow function

 return (a) => {

   // this here is lexically adopted from foo( )

   console.log(this.a);

 };

}

var obj1 = {
  a: 2,
};

var obj2 = {
  a: 3,
};

var bar = foo.call(obj1);
bar.call(obj2);          // 2, not 3!
```

```
// Without arrow function

function foo() {

 // return a normal  function

 return function f2() {
   // this here is bind to the obj2

   console.log(this.a);

 };

}

var obj1 = {

 a: 2,

};

var obj2 = {

 a: 3,

};

var bar = foo.call(obj1);
bar.call(obj2);                  // 3
```

THANK YOU !